

BURSA TEKNİK ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BİLGİSAYAR AĞLARI DERSİ DÖNEM PROJESİ TEKNİK RAPORU

Proje Başlığı: NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı, Trafik İzleme ve Ağ Performans Analiz Platformu

Geliştirici / Öğrenci: Elif İrem ŞAHİN-23360859744

1. GİRİŞ

Bilgisayar ağları mimarisinde, taşıma katmanı (Transport Layer) servisleri temel olarak iki uç protokol etrafında şekillenir: Güvenilir, bağlantı yönelimli (connection-oriented) TCP ve hızlı, bağlantısız (connectionless) UDP. TCP protokolü; akış kontrolü (Flow Control), tıkanıklık kontrolü (Congestion Control) ve kayıpsız iletim garantisi sunarken, beraberinde getirdiği el sıkışma (handshake) süreçleri ve büyük başlık (header) yükleri nedeniyle yüksek gecikme üretebilir. Öte yandan UDP, asgari başlık ek yüküyle veriyi en hızlı şekilde iletmeye odaklanır ancak paket kaybı, paketlerin sırasının bozulması veya mükerrer iletim gibi durumlara karşı herhangi bir koruma mekanizmasına sahip değildir.

Bu proje kapsamında geliştirilen **NetProbe** platformu, UDP taşıma katmanı protokolünün sunduğu düşük gecikmeli veri iletim avantajından yararlanırken, onun doğasında bulunmayan güvenilirlik mekanizmalarını **uygulama katmanında (Application Layer)** gerçekleştirmeyi hedeflemektedir. Yazılımsal olarak tasarlanan Stop-and-Wait ARQ (Automatic Repeat reQuest) akış kontrol mimarisi sayesinde, UDP soketleri üzerinden yapılan dosya aktarımlarının hatasız, sıralı ve veri bütünlüğü korunmuş bir şekilde gerçekleştirilmesi güvence altına alınmıştır.

2. PROBLEM TANIMI VE PROJE AMACI

UDP protokolü doğrudan kullanılarak ham veri akışıyla büyük dosyalar transfer edilmek istendiğinde ağ üzerinde şu kararsızlıklar yaşanmaktadır:

- Paket Kaybı (Packet Loss):** Ağdaki yönlendirici (router) kuyruklarının dolması veya hat gürültüleri nedeniyle paketlerin alıcıya ulaşamaması.
- Mükerrer Paket İletimi (Duplicate Packets):** Ağdaki gecikmeler nedeniyle zaman aşımına uğrayan paketlerin istemci tarafından yeniden gönderilmesi sonucu alıcının aynı veriyi birden fazla kez alması ve dosya içeriğinin çoklanarak bozulması.
- Veri Bozulması (Data Corruption):** Fiziksel veya bağlantı katmanındaki sinyal gürültüleri sebebiyle taşınan paket bitlerinin değişmesi.
- Sonsuz Kilitlenme (Deadlock):** İstemcinin veya sunucunun paket kayıpları anında birbirini sonsuza dek beklemesi ve sistem kaynaklarının bloke olması.

NetProbe projesinin temel amacı; bu kronik UDP problemlerini çözmek amacıyla; paket numaralandırma (Sequence Number), onaylama (ACK), zaman aşımı yönetimi (Timeout), maksimum yeniden deneme (Retransmission), veri parçacığı bütünlük denetimi (MD5 Checksum) ve uçtan uca dosya doğrulama (SHA-256) mekanizmalarını içeren, harici bir hazır ağ kütüphanesi barındırmayan özgün bir uygulama katmanı protokolü inşa etmektir.

3. SİSTEM MIMARISI VE ALGORİTMİK AKIŞ

Sistem, istemci (client.py) ve sunucu (server.py) olmak üzere iki bağımsız düğümün UDP soketleri üzerinden asenkron haberleşmesi esasına dayanır. İstemci, aktarılacak dosyayı okuyarak yapılandırılmış JSON paketlerine dönüştürür. Sunucu ise her paketi sırasıyla kabul edip doğruladıktan sonra hedef dosyaya yazar. Eğer istemci belirli bir süre (Timeout) içinde beklediği ACK paketini alamazsa, ilgili paketi ağ üzerinden yeniden iletir (Retransmission). Eğer ardışık deneme sayısı maksimum sınıra (MAX_RETRIES = 5) ulaşırsa, istemci süreci "Abort" ederek sunucuya acil durum sinyali fırlatır ve kilitlemelerini önler.

4. UYGULAMA KATMANI PROTOKOL TASARIMI

Haberleşme sürecindeki tüm mesajlaşmalar, taşınabilirlik ve esneklik amacıyla JSON (JavaScript Object Notation) formatında serileştirilerek bayt dizisine dönüştürülmüştür. Tasarlanan paket yapıları aşağıda detaylandırılmıştır:

4.1. Veri Paketi (DATA)

Dosya içeriklerinin güvenli taşınmasından sorumludur. Tip alanı "DATA" olarak set edilir.

```
{
  "type": "DATA",
  "seq": 1,
  "total_packets": 512,
  "checksum": "8f1a23b9d0b64c1251c89fef3d9412a8",
  "data": "aXN0ZWljYS1zdW51Y3UgbWltYXRpc2k...",
  "loss_rate": 0.20
}
```

- **seq:** Paketin benzersiz sıra numarası (Sequence Number).
- **total_packets:** Proje föyünde zorunlu kılınan, dosyanın toplam kaç paketten oluştuğunu sunucuya bildiren alan.
- **checksum:** Taşınan verinin bozulup bozulmadığını kontrol etmek için hesaplanan MD5 özeti.
- **data:** latin-1 formatında karakter dizisine dönüştürülmüş ham dosya verisi (Payload).
- **loss_rate:** Deney merkezi otomasyonunun sunucuda simüle edeceği anlık paket kaybı oranı parametresi.

4.2. Onay Paketi (ACK)

Sunucunun veriyi hatasız aldığını istemciye bildirdiği pakettir.

```
{
  "type": "ACK",
  "ack": 1,
  "checksum": "c4ca4238a0b923820dcc509a6f75849b"
}
```

4.3. Bitiş (END) ve İptal (ABORT) Paketleri

- **END Paketi:** Transferin bittiğini bildirir; tüm dosyanın orijinal bütünlüğünü teyit etmek amacıyla istemci tarafından hesaplanan SHA-256 hash değerini içerir.
- **ABORT Paketi:** İstemcinin 5 denemede de başarılı olamadığı paket seviyesinde transferi durdurduğunu (Abort) sunucuya bildiren acil durum kontrol paketidir.

5. GERÇEKLEME DETAYLARI

5.1. İkili (Binary) Verilerin Metin Tabanlı Kanaldan Taşınması

Dosyalar disk üzerinden ham bayt (binary - "rb") modunda okunur. Standart JSON formatı ham bayt verilerini doğrudan serileştiremediği için bit kayıplarını tamamen engelleyen ve ASCII karakter aralığını koruyan **latin-1** kodlama tablosu kullanılmıştır. İstemci bayt verisini `decode("latin-1")` ile metne çevirerek JSON'a gömer; sunucu ise gelen metni `encode("latin-1")` ile tekrar ham bayt formuna dönüştürerek diske yazar.

5.2. Sıkı Sıra ve Mükerrer Paket Kontrolü

Sunucu, her gelen paket için ardışık bir sıra denetimi yürütür. Eğer gelen paketin `seq` numarası, sunucunun beklediği sıra numarasına (`expected_seq`) eşitse veri doğrulanıp diske işlenir ve onay üretilir. Eğer gelen paket numarası beklenen değerden küçükse (`current_seq < expected_seq`), sunucu bu paketin ağdaki gecikmeden ötürü mükerrer (Duplicate) iletildiğini anlar; veriyi dosyaya tekrar yazmaz, sadece kaybolmuş olma ihtimaline karşı ilgili ACK paketini yeniden gönderir.

6. KARŞILAŞILAN İŞLETİM SİTEMI SORUNLARI VE ÇÖZÜMLERİ

1. Windows Soket Sıfırlama Hatası (WinError 10054): UDP protokolü doğası gereği bağlantısız çalışmasına rağmen, Windows işletim sistemi sunucu kapalıyken gönderilen paketlere karşılık ICMP "Port Unreachable" mesajı alırsa bir sonraki soket okumasında agresif bir şekilde istisna fırlatır. Bu kilitlenmeyi aşmak için istemci soketinde Windows'a özgü `SIO_UDP_CONNRESET` alt katman parametresi kod seviyesinde kapatılmıştır:

```
if hasattr(socket, 'SIO_UDP_CONNRESET'):
    client.ioctl(socket.SIO_UDP_CONNRESET, False)
```

2. Buffer Aşım Hatası (WinError 10040): Deneysel çalışmalar sırasında paket boyutları (Chunk Size) 4096 bayta ulaştığında, JSON ek yüküyle (overhead) birleşen paket boyutu sunucunun okuma sınırını aşmıştır. Sunucudaki alıcı buffer boyutu UDP'nin teorik üst limiti olan **65535** bayta yükseltilerek sorun çözülmüştür.

7. PERFORMANS METRIKLERİNİN MATEMATİKSEL TANIMLARI

Sistemin ağ üzerindeki verimliliğini ve başarısını kantitatif olarak ölçebilmek amacıyla kullanılan metriklerin matematiksel ifadeleri aşağıda türetilmiştir:

$$\text{Completion Time } (T_c) = T_{bitis} - T_{baslangic}$$

Ağ katmanından geçen tüm başlık ek yükleri, JSON anahtarları ve yeniden iletilen mükerrer paketler dahil toplam transfer hızı (Throughput):

$$\text{Throughput } (S_{thru}) = [\text{Toplam İletilen Bayt} \times 8] / T_c \text{ (bps)}$$

Yalnızca uygulama katmanında diske başarıyla yazılan, protokol yüklerinden ve retransmission paketlerinden tamamen arındırılmış net dosya verisinin iletim hızı (Goodput):

$$\text{Goodput } (S_{good}) = [\text{Orişinal Dosya Boyutu} \times 8] / T_c \text{ (bps)}$$

$$\text{Retransmission Rate } (R_{ret}) = [\text{Tekrar Gönderilen Paket Sayısı} / \text{Toplam İletilen Paket Sayısı}] \times 100$$

8. DENEYSEL SONUÇLAR VE TEKNİK ANALİZLER

Geliştirilen otomasyon aracı (experiment.py) vasıtasıyla 500 KB'lık test dosyası üzerinde yürütölen kontrollü deneylerin analizleri aşağıda sunulmuştur:

8.1. Deney 1: Paket Boyutu (Chunk Size) vs Throughput & Goodput

Paket boyutu 512 bayttan 4096 bayta doğru büyütöldüğünde Goodput ve Throughput değörlerinin doğrusal bir ivmeyle yükseldiğı gözlemlenmiştir. Stop-and-Wait ARQ protokol karakteristiğı gereğı, her veri paketinin ardından ağ üzerinde bir RTT (Round Trip Time) süresi boyunca boşta bekleme gerçekleşir. Küçük paket boyutlarında toplam paket adedi çok fazla olduğundan, sistem sürekli bekleme döngüsüne girer ve kanal atıllığı tavan yapar. Paket boyutunun büyütölmesi, toplam bekleme sayısını azalttığı için Kanal Kullanım Oranını (Channel Utilization) optimize etmiştir.

8.2. Deney 2: Timeout Süresi vs Completion Time

Sabit %20 paket kaybı altında timeout süresi 0.5 saniyeden 2.0 saniyeye çıkarıldığında, toplam transfer süresinin (Completion Time) katlanarak arttığı görölmüştür. Kayıplı ağ simölasyonunda paketler veya ACK'ler ağda düştüğünde, istemci bir sonraki gönderimi tetiklemek için timeout süresinin dolmasını beklemek zorundadır. Timeout süresinin gereğinden uzun seçilmesi, hata durumlarında sistemin reaksiyon süresini yavaşlatarak aktarım sürelerini dramatik biçimde uzatmaktadır.

8.3. Deney 3: Paket Kayıp Oranı vs Goodput

Yapay paket kaybı oranı %0'dan %30'a doğru artırıldığında Goodput değörlerinin sert bir çöküşle sıfıra yakınsadığı tespit edilmiştir. Kayıp oranı arttıkça retransmission paket sayısı katlanarak artmakta; bu durum ağdaki toplam bayt trafiğini (Throughput) yüksek tutsa da diske yazılan orijinal dosya girdisini değıştirmediğı için doğrudan faydalı veri hızını (Goodput) baltalamaktadır.

9. SONUÇ

Bu proje kapsamında, UDP katmanı üzerinde çalışan ve Stop-and-Wait ARQ mimarisiyle donatılmış güvenilir bir veri aktarım platformu başarıyla gerçekleştirilmiştir. Sistem, en yüksek paket kaybı ve ACK düşürme oranlarına sahip simülasyon senaryolarında dahi uçtaki SHA-256 bütünlük kontrolü testlerini %100 başarıyla tamamlamış ve verilerin kayıpsız iletilebileceğini kanıtlamıştır.

10. REFERANSLAR

1. Python Software Foundation. (2026). *Socket Programming HOWTO*. Python Documentation.
2. <https://gemini.google.com/app?hl=tr>
3. <https://chatgpt.com/>